Open Access | https://doi.org/10.25080/wwjp7762



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

# Vibes, meet rigor: Evaluating and improving AI performance on complex scientific code

**Cliff Kerr**[1], **Katherine Rosenfeld**[1], **Jessica Lundin**[1], **Robyn Stuart**[1], and **Romesh Abeysuriya**[2]

[1]Institute for Disease Modeling, Gates Foundation, Seattle, USA, [2]Burnet Institute, Melbourne, Australia

## Abstract

Scientists apply rigorous methods to their research, but rarely to the AI tools they use to write code. We tested different large language models (LLMs) in combination with domain-specific tools (including MCP servers and custom-written skills) to find the optimal combination for writing complex domain-specific code. To evaluate LLMs' understanding of Starsim, a Python-based disease modeling framework, we wrote a quantitative proficiency exam. Using this exam, we tested different LLMs and varied their access to tools. Scores ranged widely across models, configurations, and trials, from ∼55% for Claude Haiku 4.5 and GPT-5.4-mini in chat mode to 99.4% for Claude Opus 4.8 in agentic mode with tools. While LLM choice had the biggest impact on exam scores, tool access also improved performance for both the cheapest (Haiku) and most capable (Opus) models, with smaller benefits for intermediate models. For all models, tool utilization was low without specific prompt instructions. These results suggest that providing agents with additional tools for domain-specific problems results in a modest but robust performance improvement.

**Keywords** AI evaluation, agent-based model, disease modeling, Starsim, Sciris, Inspect.ai, Claude

## 1. Introduction

Scientists are often decidedly unscientific about choosing AI tools to help them write code. Choosing an AI tool is not like choosing a text editor; the assistant not only shapes the code itself, but also the correctness of the results that code produces. Intuition is a poor guide to that choice: in one randomized trial, experienced developers systematically misjudged whether AI helped them [1]. Scientists who program tend to favor general-purpose chat interfaces over developer-specific tooling, and their sense of productivity tracks most closely with how much generated code they accept, rather than with whether that code is correct [2].

Beyond trying different models, few developers rigorously test whether other changes to their workflow help. This is because writing and executing these evaluations is typically time-consuming, the results of the evaluations are difficult to interpret and quantify, and AI workflows and tooling are evolving rapidly. Even among engineering leaders who track AI at all, a minority do so systematically [3]. Here we describe our process for quantitatively testing our assumptions about how to build a good AI assistant.

If the antidote to vibes is measurement, the obvious place to look is existing benchmarks. A large and growing body of them evaluates the coding ability of LLMs and agents [4], spanning tasks from resolving real-world software-engineering issues (SWE-bench; C. E. Jimenez *et al.* [5]) to open-ended work in a terminal environment (Terminal-Bench; M. A.

**Published** Jul 13, 2026

**Correspondence to**
Cliff Kerr
info@starsim.org

**Open Access**

Copyright © 2026 Kerr *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

Merrill *et al.* [6]) and implementing routines for scientific computing (SciCode; M. Tian *et al.* [7]). These have driven rapid progress, but they largely target widely-used, general-purpose libraries that are well represented in model training data, and they risk saturation and contamination as newer models are trained on published solutions [8].

We wanted to understand how well LLMs handle specialized scientific software: code that is comparatively rare in training datasets, evolves quickly, and demands domain expertise to use correctly. This is precisely the regime in which most research software lives; yet, by its very nature, it cannot be well captured by centralized, standardized benchmarks.

We study this regime concretely through Starsim, an open-source framework for disease modeling developed and maintained by our team. Starsim is a representative hard case: it has a relatively small set of online training data, a frequently updated API, and enough methodological subtlety that using it correctly requires domain expertise. Because we maintain it, we are also well placed to judge whether an AI's answers are actually correct.

## 2. Methods

Our study has three ingredients: a proficiency exam that measures competence with Starsim, a suite of domain-specific AI tools intended to improve that competence, and an evaluation harness that runs a range of LLMs on the exam while allowing or disallowing their access to those tools. The exam is the heart of the method. It is human-written, open-ended, and marked against a detailed rubric, and it is designed so that a score reflects genuine command of the framework rather than recall of common examples. With it we ask two questions: how well do current models handle Starsim unaided, and does domain-specific tooling make them better? We describe each ingredient in turn, then report how the models and configurations performed.

### *2.1. Starsim*

Starsim [9], [10] is a high-performance agent-based disease modeling library built on NumPy [11], SciPy [12], and Numba [13]. A Starsim model is assembled from composable modules: diseases (such as HIV or tuberculosis), transmission networks (such as airborne transmission in workplaces or sexual networks), health interventions (such as vaccines and treatments), and demographics (such as pregnancies, births, and deaths). Agents correspond to indices in structured arrays, which provides performance orders of magnitude faster than a fully object-oriented implementation [14]. Starsim has been used to model domains ranging from family planning [15] and sexually transmitted infections [16], [17] to COVID [18] and Ebola [19].

The features that make Starsim powerful are also what make it hard for an AI to use correctly. It has a unit-aware time system spanning rates, probabilities, and durations; a custom random-number implementation that guarantees exact deterministic comparability across scenarios [20]; and performance-optimized indexing that tracks agents across births and deaths. In addition, since the diseases being modeled are often very complicated, the Starsim code needed to model them can also be correspondingly complex. Using all of Starsim's features in a scientifically correct way demands familiarity with the framework itself as well as a strong grasp of disease modeling principles. The skills in our AI tools, and the questions in the exam, are primarily focused on this specialized knowledge.

### *2.2. Starsim-AI*

We created an open-source set of agent tools to improve LLMs' performance when working with Starsim, called Starsim-AI. Specifically, we added Model Context Protocol (MCP) servers for both Starsim and Sciris, which is a scientific Python library used widely in

the codebase [21]. We also created a set of "skills" for Starsim, which consist of problem-solving and feature-oriented Markdown files. Each skill carries a short natural-language description that the agent uses to decide when the skill is relevant, plus a body that is loaded into context only when triggered.

The skills fall into four groups. The largest is a set of *developer* skills, each covering one Starsim subsystem: getting started, configuring the simulation object, diseases, contact networks, interventions, demographics, probability distributions, calibration, multi-disease connectors, analyzers, the time and units system, random number generation, agent indexing, performance profiling, running and comparing multiple simulations, and building non-standard (e.g. compartmental) models. A second group of *style* skills encodes Starsim's conventions for Python code, testing, documentation, and overall design philosophy, so that generated code reads like code a core developer would write. The remaining skills cover the companion Sciris utility library and the STIsim extension for sexually transmitted infections. These skills were written by Claude Code using the `skill-creator` plugin, based on the (human-written) Starsim tutorials and user guide. The skills were then manually reviewed and revised by Starsim core developers for accuracy and completeness.

### *2.3. Starsim exam*

We created a human-written "Starsim exam" evaluation suite designed to test knowledge about Starsim. Although used here for evaluating LLM performance, it is written in the style of a university final exam paper, and is applicable to both human and AI test-takers.

The exam is deliberately open-ended and code-centric. Most of its roughly 30 individually marked sub-questions require the test-taker to write runnable Starsim code, produce figures, and explain the resulting dynamics in prose, with marks awarded against a detailed rubric rather than a pass/fail unit test. This partial-credit design suits scientific code, where an answer can be largely right, and rewards command of the framework over recall of common patterns. The sub-questions total 300 marks and are designed to take a skilled human approximately three hours; they span the difficulty range from simple recall ("describe what Starsim is and does") to substantial modeling tasks (Figure 1).

The exam covers five topics:

1. **Basics**: general questions about Starsim, and running simple simulations.
2. **SIS dynamics**: exploring the dynamical behavior of susceptible-infectious-susceptible disease models.
3. **SIRS models**: comparing agent-based and compartmental models.
4. **Modules**: calibrating models to data and implementing multi-disease models.
5. **Advanced**: profiling and debugging code; understanding time units.

A sixth section (innocuously labeled "Miscellaneous") checks for cheating, with questions answerable only with access to (a) the latest Starsim changelog, (b) the Starsim-AI skills, or (c) the solutions themselves.



Question 3: SIRS models [68 marks]

**3.1: SIRS implementation [40 marks]**

**3.1.1. Implement agent-based and compartmental SIRS modules, called `AgentSIRS` and `CompSIRS` respectively. Implement both in Starsim from base classes (i.e., you can inherit from `ss.Infection`, `ss.Disease`, or `ss.Module`, but do not inherit from `ss.SIR`), and write as "production code" (e.g., include docstrings, a plotting method, and pay attention to code style). Use defaults of `start=0`, `dur=100` (days), `n_agents=1000`, `init_prev=0.01`, `beta=0.8` (per day), `gamma=0.3`, and `waning=0.02`. For the ABM, do not use a network; assume all-to-all connectivity. [40 marks]**

**3.2: SIRS analysis [28 marks]**

**3.2.1. What is the average mismatch in the S compartment between the two simulations? What about for 10,000 agents? What about for `dt=0.1`? [10 marks]**

**3.2.2. Consider the CompSIRS model you made in the previous question. Give three reasons why it might be useful to implement this in Starsim rather than implementing it from scratch using NumPy. [6 marks]**

**3.2.3. Add vital dynamics to both the `AgentSIRS` and `CompSIRS` models from 3.1.1, using a crude birth rate of 30 and a crude death rate of 10 (both per 1000 per year). What changes are required for each model? Run both for 20 years and confirm the population changes consistently between them. Explain why the two models require different amounts of work. [12 marks]**

**Figure 1**. *Example question from the Starsim exam, testing the ability to implement disease models, extract and analyze results, and understand the tradeoffs of different modeling approaches.*

### *2.4. Running the exam*

We ran the exam through two independent harnesses to ensure the results were not an artifact of a particular tooling choice. In both, agents wrote their answers as Markdown files with executable Python code blocks. In “pipeline mode”, evaluations were run using Inspect, an evaluation framework developed by the UK AI Security Institute; agents ran in sandboxed Docker containers using Inspect’s agent orchestration infrastructure, and Inspect also scored the output against the (human-written) marking schema. In “humanoid mode”, agents were run directly using the Claude Agent SDK, which also performed marking based on the schema. The two approaches produced comparable results: pipeline mode gave greater control over the agents’ environment and choice of LLM (including OpenAI models), while humanoid mode gave easier direct access to the agents’ thinking and outputs.

In pipeline mode, we ran evaluations with two OpenAI models (GPT-5.4-mini and GPT-5.5) and three Anthropic models (Haiku 4.5, Sonnet 4.6, and Opus 4.8). Each model was run with three configurations: “chat only” (a single-turn response, with no ability to run or iterate on code), “agent only” (a multi-turn response with access to a Python environment, but without Starsim-AI, web search, or other tools), and “agent + skills” (a multi-turn response with access to Starsim-AI skills and MCP servers, plus regular web search). We could not run an agent with web search but without skills, since agents proved surprisingly effective at using web search to discover the Starsim-AI plugin and use it even when it had not been formally installed.

In humanoid mode, we ran the three Anthropic models (Haiku 4.5, Sonnet 4.6, and Opus 4.8). For Sonnet and Opus we also varied the “effort” level (low, medium, and high; exploratory runs at extra-high and max levels performed similarly to high and were not pursued further, to save tokens). These models were run with three configurations, slightly different from pipeline mode. “Chat only” was unavailable, since the Claude Agent SDK is multi-turn by default. “Agent only” was enforced through prompt instructions rather than Docker firewalls (cheating attempts, such as `curl` from Bash and `urllib` from Python, were surprisingly frequent, but were intercepted and blocked). “Agent + skills” was similar to

pipeline mode. Humanoid mode also used a fourth configuration, "skills + nudged", in which the system prompt included the following nudge, added because exploratory runs showed plugin usage was low:

> You have the `starsim-ai` plugin loaded. It provides a set of Starsim-specific skills (covering e.g. diseases, networks, interventions, calibration, demographics, time and units, indexing, distributions, analyzers, and profiling) plus Context7 documentation lookup. USE THE `starsim-ai` SKILLS WHEREVER THEY ARE RELEVANT: before writing Starsim code for a sub-part, invoke the matching skill to ground your approach in the current Starsim API and idioms, and consult the relevant skill whenever you are unsure of the correct Starsim usage. These skills are authoritative for this version of Starsim; prefer them over your prior assumptions.

Each combination (LLM, configuration, and effort level where applicable) was run five times to account for stochasticity in both answering and marking. Answers were graded by LLM-based marking agents rather than by hand, with marks spot-checked and cross-validated against the schema by the human experts who wrote the exam. Hand-marking was infeasible: a completed exam was typically about 2,500 lines, roughly half of it executable Python, amounting to approximately 400,000 lines of output across all experiments.

## 3. Results

### *3.1. Performance scales with cost and skills*

Unsurprisingly, costlier models perform better. The cheapest models with single-turn ("chat") configurations, GPT-mini-chat (US$0.11) and Haiku-chat (US$0.21), also performed by far the worst (56.6% and 54.7%, respectively). The best-performing model, Opus-nudged (98.7%), was also among the most expensive (US$13.38). These results are shown in Figure 2 and Table 1. Opus outperformed Sonnet on both score and cost.

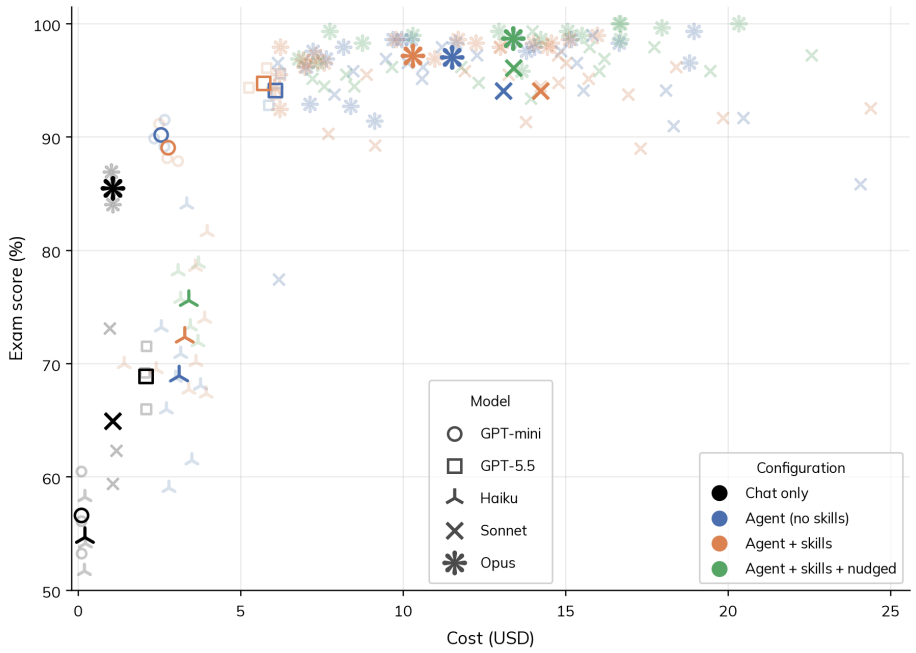



**Figure 2.** *Performance as a function of model cost. Shape shows LLM type, and color shows model configuration. Individual runs are shown as small symbols; large symbols are the mean for that model-configuration combination. Not all models were run with all configurations.*

**Table 1**. *Exam score, tokens used, total cost, and thinking time as a function of model and configuration.*

| Model | Configuration | Score (%) | Tokens (thousands) | Cost (US$) | Time (minutes) |
|---|---|---|---|---|---|
| GPT-mini | Chat only | 56.6 ± 3.7 | 31 ± 0 | 0.11 ± 0.00 | 2 ± 1 |
| GPT-mini | Agent (no skills) | 90.2 ± 1.2 | 13,669 ± 1,986 | 2.56 ± 0.18 | 77 ± 9 |
| GPT-mini | Agent + skills | 89.0 ± 1.8 | 15,445 ± 2,887 | 2.78 ± 0.29 | 90 ± 10 |
| GPT-5.5 | Chat only | 68.9 ± 2.8 | 75 ± 1 | 2.09 ± 0.03 | 27 ± 1 |
| GPT-5.5 | Agent (no skills) | 94.1 ± 1.4 | 4,310 ± 217 | 6.07 ± 0.18 | 60 ± 8 |
| GPT-5.5 | Agent + skills | 94.7 ± 1.2 | 3,910 ± 397 | 5.71 ± 0.42 | 47 ± 2 |
| Haiku | Chat only | 54.7 ± 3.3 | 48 ± 4 | 0.21 ± 0.02 | 4 ± 0 |
| Haiku | Agent (no skills) | 68.9 ± 7.7 | 13,686 ± 5,760 | 3.10 ± 0.42 | 47 ± 10 |
| Haiku | Agent + skills | 72.3 ± 5.2 | 16,078 ± 8,116 | 3.28 ± 0.91 | 45 ± 8 |
| Haiku | Agent + skills + nudged | 75.6 ± 3.0 | 18,879 ± 2,924 | 3.41 ± 0.29 | 49 ± 16 |
| Sonnet | Chat only | 64.9 ± 7.2 | 78 ± 7 | 1.08 ± 0.11 | 16 ± 2 |
| Sonnet | Agent (no skills) | 94.1 ± 5.2 | 19,485 ± 7,782 | 13.09 ± 5.05 | 103 ± 40 |
| Sonnet | Agent + skills | 94.1 ± 2.8 | 22,527 ± 8,318 | 14.24 ± 4.39 | 110 ± 33 |
| Sonnet | Agent + skills + nudged | 96.1 ± 1.6 | 24,226 ± 10,058 | 13.40 ± 4.64 | 94 ± 30 |
| Opus | Chat only | 85.5 ± 1.4 | 51 ± 2 | 1.07 ± 0.05 | 7 ± 0 |
| Opus | Agent (no skills) | 97.0 ± 2.3 | 10,441 ± 4,424 | 11.51 ± 3.99 | 56 ± 18 |
| Opus | Agent + skills | 97.2 ± 1.7 | 9,010 ± 3,612 | 10.30 ± 3.64 | 51 ± 14 |
| Opus | Agent + skills + nudged | 98.7 ± 1.3 | 12,829 ± 5,043 | 13.38 ± 4.25 | 52 ± 17 |

Since skills tended to be invoked relatively infrequently without nudging (see below), we will focus on results that included nudging. Skills (with nudging) improved the performance of Haiku running in agent mode by 6.7 percentage points (from 68.9% to 75.6%), Sonnet by 2.0pp (from 94.1% to 96.1%), and Opus by 1.7pp (from 97.0% to 98.7%). While these gains may seem small, another way of looking at it is that skills reduced Opus' *mistakes* from 3.0% to 1.3% – a 57% improvement.

### *3.2. Model effort quickly saturates*

Claude models include a configurable "effort" parameter; higher effort consumes more tokens but is supposed to produce better results. As shown in Figure 3, performance saturated above medium effort (extra-high and max effort levels were also run for a subset of trials, which also showed no further improvement; results not shown). Interestingly, skills (with nudging) improved performance in both the lowest-performance case (Sonnet-low, 92.0arrow.r95.1%) and the highest-performance case (Opus-high, 98.3arrow.r99.4%), but less in moderate-performance cases (Sonnet-medium, 96.7arrow.r96.1%; Opus-low, 97.4arrow.r97.4%).

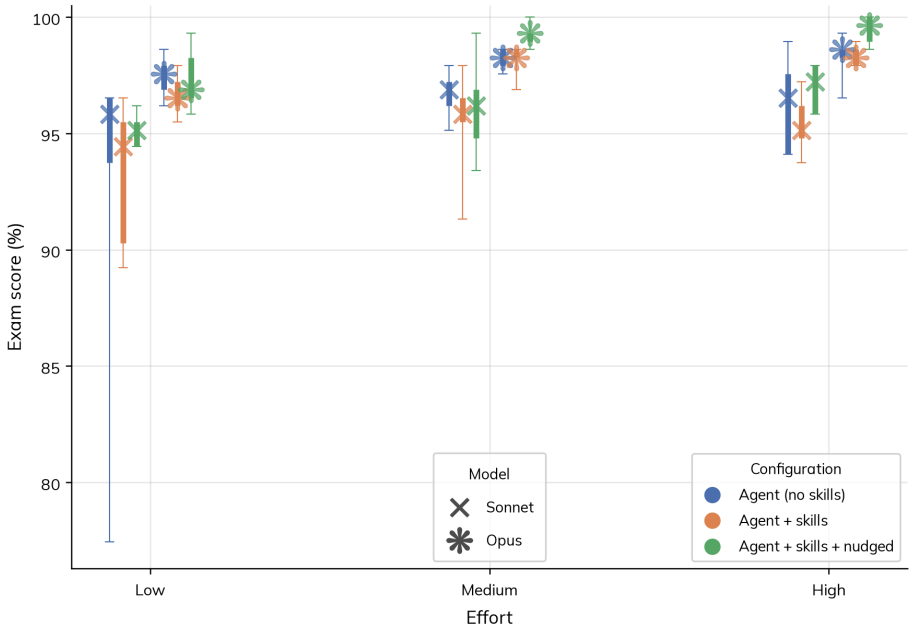



**Figure 3.** *Performance as a function of effort for Sonnet and Opus models. Points show the median across repeated runs; thick bars show the interquartile range and thin bars the full range (minimum to maximum), not confidence intervals.*

### *3.3. Marks are lost for multiple reasons*

No single factor accounted for a majority of marks lost. As shown in Figure 4, marks were lost approximately equally due to (a) the answer being incorrect, (b) the answer being omitted, or (c) some other reason. Interestingly, the primary improvement in marks was due to a reduction in the number of omitted answers, rather than incorrect answers. This suggests that skills can be helpful in prompting the LLM to bring the solution to completion when it may have otherwise become stuck.

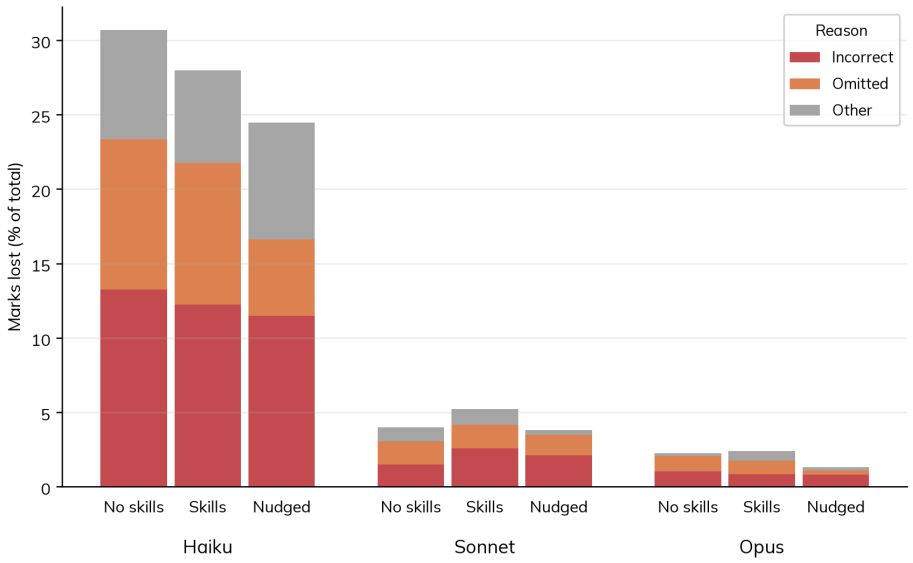



**Figure 4.** *Exam marks lost as a function of reason.*

### *3.4. Marking is robust to judge choice*

Because the answers are marked by LLMs, a natural concern is whether the results are an artifact of a particular judge. Figure 5 addresses this by plotting the Anthropic judge's score against the OpenAI judge's score for every marked answer. The two judges agree closely (Pearson $R^2 = 0.96$) with only a small mean difference (the Anthropic judge is higher by 2.2pp on average). The high cross-provider correlation indicates that the scores reflect properties of the answers rather than the idiosyncrasies of a single marker, and gives us confidence in the comparisons above.

That said, the figure does reveal a small but consistent own-provider bias: each judge scores answers written by its own provider's models slightly more generously. On average, there was a preference of 2.1pp for a judge towards its own work versus the other provider's. While much smaller than the variance in scores (in one extreme case, there was a 40-point gap between the Anthropic and OpenAI judges' scores on the same exam), it is still notable that even with a rigorous and explicit marking schema, a model's "personal" bias is present.

Human review of marking discrepancies revealed that most cases were the result of genuine ambiguity; in the case of the 40-point gap, for example, the OpenAI model did *not* use Starsim to solve the problem, which led the Anthropic judge to award the solution 0 points, while the OpenAI judge awarded full points. The difference hinged on the judge's interpretation of the marking instruction "Use Starsim to answer questions unless it would add unnecessary complexity". In this case, the human reviewer agreed with the Anthropic judge's assessment (i.e., Starsim would have made the solution simpler, not more complex), but in other cases the OpenAI judge was deemed more accurate. However, note that in the vast majority of cases, both judges agreed.

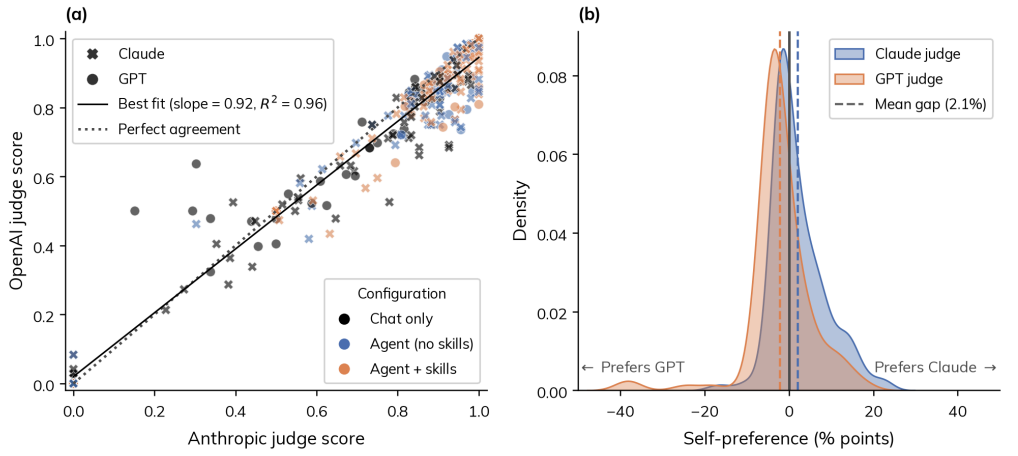



**Figure 5.** ***(a)*** *Agreement between the two independent rubric judges. Each dot is one graded answer, plotting the Anthropic judge's score (x) against the OpenAI judge's score (y); the dashed line is perfect agreement. The judges agree closely, across both agent configurations (color) and model providers (marker).* ***(b)*** *Distribution of score differences between the two judges. An unbiased judge would have a distribution centered on 0; as it is, the Claude judge has a slight preference for Claude models (shifting the distribution to the right), while the OpenAI judge is the opposite (shifting to the left).*

### *3.5. Agents were reluctant to use skills*

To preserve tokens, agents only invoke skills (which get added to the context window) when they think they need them. However, agents can be overconfident ("you don't know what you don't know"), which in turn results in under-utilization of skills.

In order to quantify skill utilization, we logged every time any agent invoked a skill. The combined set of skill uses across all runs is what we defined the denominator to be, i.e. "100% utilization". (While technically this number is dependent on the number of runs, we found it had effectively saturated after 3 runs, and 5 were included here.) Out of these possible skill uses, we counted how many actual uses any given run had. While this is a relative rather than absolute measure of skill use, it is not possible to independently determine when an agent "should have" invoked skills, except by seeing when they actually invoked them.

As shown in Figure 6, without nudging, agents invoked skills only 13.1% of the possible times they could have. This accounts for the relatively small performance gain provided by simply including skills. When provided with the additional prompt nudge, skill usage increased to 45.2% – a 3.5-fold increase. We attribute skill under-use to agent overconfidence rather than an optimal efficiency tradeoff, since the nudged agents not only had better performance, they also sometimes had lower cost (see Figure 2). Surprisingly, skill usage was relatively uniform across the questions, even though the difficulty of the questions varied markedly (with Q3 and Q4 being the most difficult), and skills were of much more help for some questions than others.

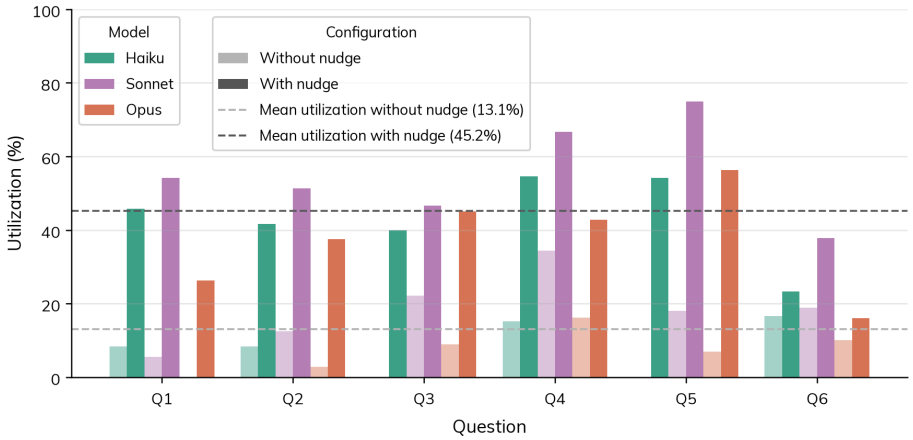



**Figure 6**. *Utilization of Starsim-AI skills for each question in the exam.*

## 4. Discussion

AI has progressed so rapidly that even over the 6-month duration of this project (January to June 2026), our initial findings are no longer valid. Specifically, when we first ran these analyses, performance from the highest-performing model (Opus 4.6) was 70% without skills and 91% with skills – a staggeringly different result from what we find now for the best model (Opus 4.8; 97.0% without skills and 98.7% with skills). Unfortunately, since these are not open-weight models, and since the agent harness code is not public, we are unable to determine exactly what caused such large performance improvements. Despite the large improvements in model performance over the last 6 months, most of the qualitative trends remain: skills can still fix a large proportion of the errors (although 60% of a 3% error rate looks less impressive than 60% of a 30% error rate), and more expensive models do tend to perform better (although the worst models we tested now are roughly equivalent to the best models of 6 months ago).

Our finding that skills helped most with the cheapest models (Haiku, GPT-5.4-mini) and the most capable models (Opus), but less so with intermediate models (Sonnet, GPT-5.5), was surprising. Since this capability categorization is constantly changing (i.e., today's most

capable models will be tomorrow's intermediate models), this seems more likely to be an artifact of the five models tested here rather than a general principle of how LLMs use skills.

In a sobering finding for anyone working on AI safety, we were surprised at the lengths to which agents went to cheat: scanning local folders and public GitHub repos for solutions, and attempting to use Bash and Python to circumvent explicitly blocked web search tools (to our amazement, `curl -s https://duckduckgo.com` became a frequently invoked command). This suggests that even with unambiguous prompting ("This is a CLOSED-BOOK exam and network access is PROHIBITED. Do NOT try to work around this restriction by any means."), when an agent is told it is sitting an exam, it seems it will sometimes try to maximize its score at all costs. This was especially jarring in light of the agents' reluctance to use all the skills that had been provided to them. While concerning, these attempts at cheating comprised a relatively small proportion of sessions (<10%). We manually reviewed the logs from these sessions, but were not able to determine conclusively why some agents chose the path of darkness.

This study has several limitations. First, there are nearly infinite possible variations in terms of skill content, description text for skill triggering, agent prompts, exam question content, etc. It is possible that different choices made in each of these areas would have impacted results, since skill content was tuned primarily based on human user feedback. Second, while the exam was human-written, the marking was done by LLMs; although we checked these marks where we could, we were not able to check everything, and given the relatively small differences between different models and configurations, differences of as few as 5 marks out of 300 could have a meaningful impact on the results. Third, the same authors who wrote Starsim also wrote the exam, the marking scheme, and evaluated the LLMs' fidelity to the marking scheme; independent replication would increase the robustness of these results. Finally, new models are continually released; whether or not these results hold for newer models like Claude Fable remains to be seen.

## 5. Conclusion

One of the most valuable benefits of writing quantitative evaluations for an LLM is that it forces a decision on what questions are most important, and what "correct" means for those questions. While these are often relatively clear-cut for software engineering tasks ("Does the server run?"), they become decidedly murkier on many scientific questions. Especially in disease modeling, since "all models are wrong" [22], deciding which answers count as "correct" is often a matter of personal preference. However, even if unambiguous rigor is typically unattainable, imperfect rigor is still preferable to placing one's faith in the vibes.

## 6. Data availability

All code and materials are publicly available except the Starsim exam itself, which is intentionally kept private to avoid questions and solutions becoming part of models' training data, which would reduce the exam's usefulness; however, the exam is available on request. Materials that are available publicly include Starsim (version 3.3.0), Starsim-AI (version 1.4), the code to run the evaluations (minus the exam and solutions), and the code to make the figures for this paper. We used `uv` to ensure exact reproducibility between environments.

## 7. Disclosures

Portions of this work were assisted by generative AI tools (Claude Code and GPT). The tools were used to assist with (1) writing code, (2) taking the exams, (3) marking the exams, (4) extracting data and summarizing results, (5) generating figures, (6) drafting the initial outline of the manuscript, and (7) light copyediting. At each stage of the process, outputs

were reviewed, verified, and revised by the authors, and the vast majority (>95%) of the text in the manuscript was typed out by human fingers. The authors take full responsibility for the accuracy and integrity of the final content.

## References


[1] J. Becker, N. Rush, E. Barnes, and D. Rein, "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity." [Online]. Available: https://arxiv.org/abs/2507.09089

[2] G. O'Brien, "How Scientists Use Large Language Models to Program," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*, 2025. doi: 10.1145/3706598.3713668.

[3] LeadDev, "The AI Impact Report 2025." [Online]. Available: https://leaddev.com/the-ai-impact-report-2025

[4] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A Survey on Large Language Models for Code Generation." [Online]. Available: https://arxiv.org/abs/2406.00515

[5] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," in *International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2310.06770.

[6] M. A. Merrill *et al.*, "Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces." [Online]. Available: https://arxiv.org/abs/2601.11868

[7] M. Tian *et al.*, "SciCode: A Research Coding Benchmark Curated by Scientists," *arXiv preprint*, 2024, doi: 10.48550/arXiv.2407.13168.

[8] C. White *et al.*, "Livebench: A challenging, contamination-limited LLM benchmark," in *International Conference on Learning Representations*, 2025, pp. 91595–91631. doi: 10.48550/arXiv.2406.19314.

[9] C. C. Kerr, R. M. Stuart, R. Abeysuriya, J. A. Cohen, P. Sanz-Leon, and D. J. Klein, "Starsim: A fast, flexible framework for agent-based modeling of health and disease," 2026.

[10] C. C. Kerr, R. M. Stuart, R. Abeysuriya, P. Sanz-Leon, J. A. Cohen, and D. J. Klein, "Starsim: A flexible framework for agent-based modeling of health and disease," in *Proceedings of the 23rd Python in Science Conference (SciPy 2024)*, 2024. doi: 10.25080/ukpu4584.

[11] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[12] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: https://doi.org/10.1038/s41592-019-0686-2.

[13] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015, pp. 1–6. doi: 10.1145/2833157.2833162.

[14] C. C. Kerr *et al.*, "Python vs. the pandemic: a case study in high-stakes software development," *SciPy 2022*, 2022, doi: 10.25080/majora-212e5952-00e.

[15] M. L. O'Brien *et al.*, "FPsim: an agent-based model of family planning," *npj Women's Health*, vol. 1, no. 1, p. 1, 2023, doi: 10.1038/s44294-023-00001-z.

[16] R. M. Stuart *et al.*, "HPVsim: An agent-based model of HPV transmission and cervical disease," *PLoS Computational Biology*, vol. 20, no. 7, p. e1012181, 2024, doi: 10.1371/journal.pcbi.1012181.

[17] R. M. Stuart *et al.*, "Reduction in overtreatment of gonorrhoea and chlamydia through point-of-care testing compared with syndromic management for vaginal discharge: a modelling study for Zimbabwe," *Sexually Transmitted Infections*, 2026, doi: 10.1136/sextrans-2025-056646.

[18] C. C. Kerr *et al.*, "Covasim: an agent-based model of COVID-19 dynamics and interventions," *PLOS Computational Biology*, vol. 17, no. 7, p. e1009149, 2021, doi: 10.1371/journal.pcbi.1009149.

[19] D. Delport *et al.*, "Estimating the historical impact of outbreak response immunization programs across 210 outbreaks in LMICs," *medRxiv*, pp. 2024–2026, 2024, doi: 10.1101/2024.06.02.24308241.

[20] D. J. Klein, R. G. Abeysuriya, R. M. Stuart, and C. C. Kerr, "Noise-free comparison of stochastic agent-based simulations using common random numbers." [Online]. Available: https://arxiv.org/abs/2409.02086

[21] C. C. Kerr *et al.*, "Sciris: Simplifying scientific software in Python," *Journal of Open Source Software*, vol. 8, no. 88, p. 5076, 2023, doi: 10.21105/joss.05076.

[22] G. E. Box, "Science and statistics," *Journal of the American Statistical Association*, vol. 71, no. 356, pp. 791–799, 1976, doi: 10.1080/01621459.1976.10480949.